

Actividad de Clase: Gestión de Base de Datos PostgreSQL

Una tienda necesita gestionar datos de productos y ventas. Debes crear la base de datos y sus tablas en PostgreSQL (usando pgAdmin), insertar registros manualmente y desde CSV, actualizar stock, eliminar de forma segura un producto inactivo y validar con una consulta JOIN. Entregarás evidencias de cada paso.

Objetivos y Materiales de la Actividad

Manipulación de Registros

Manipular registros de una tabla con SQL (INSERT, UPDATE, DELETE) en PostgreSQL para resolver un requerimiento.

Carga de Datos CSV

Cargar datos desde un archivo CSV con utilidades del motor/pgAdmin.

Consultas JOIN

Ejecutar una consulta JOIN para verificar la integración entre tablas relacionadas.

Materiales y preparación previa (checklist)

- PostgreSQL y pgAdmin instalados y funcionando.
- Credenciales de acceso a un servidor/local de PostgreSQL.
- Archivo productos.csv con columnas (nombre, precio, stock). Si no dispones del archivo, solicita la plantilla al docente.
- Espacio para guardar capturas de pantalla de cada hito.

Requisitos técnicos y datos (definiciones breves)

- **pgAdmin**: interfaz gráfica para administrar PostgreSQL (crear BD, tablas y ejecutar SQL).
- **CSV**: archivo de texto con valores separados por comas; debe coincidir en tipos y orden de columnas con la tabla destino.

Instrucciones Paso a Paso (Parte 1)

01

Crear la base de datos y usuario (si aplica)

- En pgAdmin: Create → Database... con nombre tienda.
- (Opcional) Crear rol de trabajo y otorgar CONNECT.
- **Micro-check:** Verifica que tienda aparece en Databases.

02

Crear tablas

Ejecuta en Query Tool (ajusta si ya existen):

```
CREATE TABLE productos (  
  id_producto SERIAL PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  precio NUMERIC(10,2) NOT NULL CHECK  
    (precio > 0),  
  stock INT DEFAULT 0  
);  
CREATE TABLE ventas (  
  id_venta SERIAL PRIMARY KEY,  
  id_producto INT REFERENCES  
    productos(id_producto),  
  cantidad INT CHECK (cantidad > 0),  
  fecha TIMESTAMP DEFAULT  
    CURRENT_TIMESTAMP  
);
```

Micro-check: Refresca el esquema y valida columnas, PK y FK.

03

Insertar registros manuales

Ejecuta:

```
INSERT INTO productos (nombre,  
  precio, stock)  
VALUES ('Laptop', 750000, 10),  
('Mouse', 12000, 50);
```

Micro-check: SELECT * FROM productos; muestra 2 filas.

Instrucciones Paso a Paso (Parte 2)

01

Cargar datos desde CSV (Import/Export de pgAdmin)

- Menú de tabla productos → Import/Export Data... → Archivo productos.csv, delimitador ,, Header activado.
- **Micro-check:** SELECT COUNT(*) FROM productos; aumentó el total y no hay errores de tipo.

03

Eliminar de forma segura

Opción A (borrado físico, con precaución):

```
DELETE FROM productos
WHERE id_producto = 2;
```

Opción B (soft delete): agregar/usar columna activo BOOLEAN y marcar FALSE.

Micro-check: Verifica con SELECT que el registro fue eliminado o inactivado.

02

Actualizar stock

Disminuye stock del id 1 como simulación de venta:

```
UPDATE productos
SET stock = stock - 2
WHERE id_producto = 1;
```

Micro-check: Confirma el cambio con SELECT stock FROM productos WHERE id_producto=1;

Nota de ampliación: Usa siempre WHERE para evitar actualizar todos los registros por error.

04

Consulta de validación (JOIN)

Inserta una venta de ejemplo y consulta:

```
INSERT INTO ventas (id_producto, cantidad) VALUES (1, 2);
SELECT p.nombre, v.cantidad, v.fecha
FROM ventas v
JOIN productos p ON v.id_producto = p.id_producto;
```

Micro-check: La consulta devuelve nombre del producto, cantidad y fecha.

Criterios de Evaluación y Entrega

Criterios de éxito (evidenciables) y errores comunes

- Entregas capturas claras de: creación de BD, tablas, INSERT, importación CSV, UPDATE, DELETE/soft delete y JOIN funcionando.
- Datos cargados sin errores de tipo (coincidencia columnas/CSV). **Error común:** desalinear orden o tipos en COPY/Import.
- Uso correcto de WHERE en UPDATE/DELETE. **Error común:** omitir WHERE y afectar toda la tabla.
- Aplicación de CHECK/PK/FK como se definió en el modelo.

Formato de entrega y tiempos de desarrollo

- **Entregable:** 1 PDF con capturas y, al final, bloque de SQL ejecutado (copiado desde Query Tool).
- **Tiempo total:** 45'.
- **Modalidad:** Individual (asincrónica).

Criterio (ponderación)	Excelente	Competente	Básico	Insuficiente
1. Comprensión del concepto clave (25%)	Explica con precisión el rol de PostgreSQL/pgAdmin, DML (INSERT/UPDATE/DELETE) y riesgos/soft delete; justifica cada decisión.	Describe correctamente los conceptos y precauciones más relevantes.	Menciona algunos conceptos, con vacíos o imprecisiones.	Confunde conceptos o ignora riesgos.
2. Aplicación técnica: correctitud y reproducibilidad (35%)	BD y tablas creadas con PK/FK/CHK; inserciones manuales y desde CSV sin errores; UPDATE/DELETE con WHERE o soft delete; JOIN operativo. Evidencias completas.	Implementa la mayoría de tareas con pequeños errores corregidos.	Varias tareas incompletas o con errores persistentes.	No logra ejecutar tareas clave.
3. Calidad del entregable (20%)	PDF ordenado: capturas legibles por paso + bloque de SQL final. Nombres/fechas legibles.	PDF con la mayoría de capturas y SQL.	PDF poco claro o incompleto.	Entrega ausente o ilegible.
4. Justificación/interpretación de resultados (20%)	Argumenta por qué eligió DELETE vs. soft delete; verifica tipos/validación post-import y lectura del JOIN.	Justifica parcialmente y verifica algunos resultados.	Escasa justificación; verifica superficialmente.	No justifica ni verifica.

Ponderaciones totales: 100% (25% + 35% + 20% + 20%)